SBA308a 2025.01.12
README_SBA308a

We apply sec.gov APIs to extract board of director names for a specific company, which is our proximal endpoint.

1. The sec.gov API is detailed and extremely well documented, but like most government services the documentation is impenetrable, not entirely correct, and most likely varies over time. It really ought to be laid out in a user-friendly format.

All data at sec.gov is public and freely available; posting on the other hand involves strict requirements.

As I looked around for tools that could achieve our endpoint, it emerged that they are largely proprietary, require an API key, and incur usage charges. The latter is often not evident at first glance, and only once one has studied the API documentation in detail. I have the sense this is intentional. I don't know if there is any freeware for this purpose, and if not, I can see why not. If the documentation were as good as many of the APIs Josh and Bryan recommended, I'd guess that freeware would be readily available.

I used chatGPT not only for research about what my endpoint should be, and also for how to code and implement it. The code that chatGPT wrote does not work in my hands, most likely because the directory structure changed since the last time chatGPT's database was updated.

My primary effort has been invested in elucidating the directory structure of the sec.gov website and revising the code that chatGPT wrote to properly account for this directory structure. Although these revisions are not complete and the code works only partly, it is obvious that now that I've figured out the directory and file structure, I can get the code to work properly within a few days.

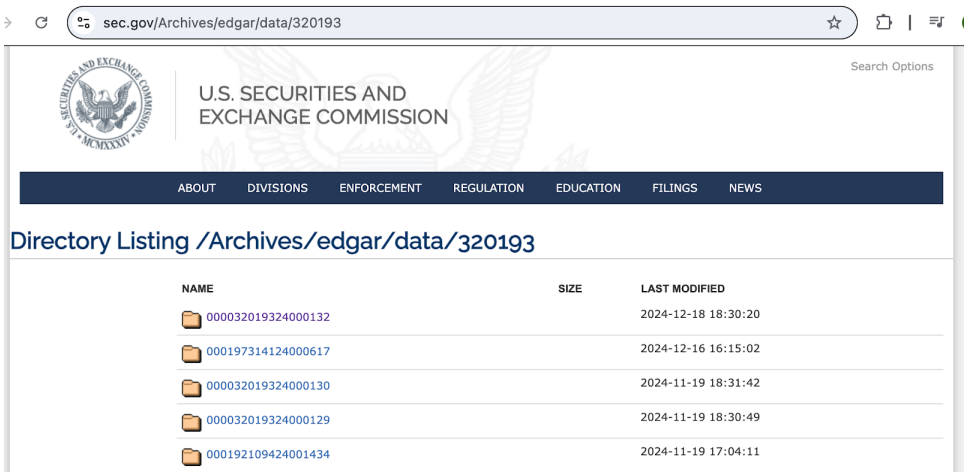
I've attached (or will attach) the chatGPT transcripts so that the reader understands what information chatGPT provided.

2. Directory and file structure:

https://www.sec.gov/	a home page
https://www.sec.gov/Archives	oops page not found
https://www.sec.gov/Archives/	URL /Archives/ not found on this server
https://www.sec.gov/Archives/edgar/	Not Found
https://www.sec.gov/Archives/edgar/data/	Not Found:

"The requested URL /Archives/edgar/data/ was not found on this server.
Additionally, a 404 Not Found error was encountered while trying to use an ErrorDocument to handle the request. *Apache Server at www.sec.gov Port 80"*

https://www.sec.gov/Archives/edgar/data/320193 for cik = '0000320193" (Apple's CIK) yields a directory listing as seen below:



Within each of these "CIK subdirectories" such as for example, https://www.sec.gov/Archives/edgar/data/320193/000130817925000009 found in https://www.sec.gov/edgar/browse/?CIK=320193&owner=exclude, we want the "defa14a" form submissions, e.g. "defa14a.htm" .

These apparently correspond to the "proxy statements" and/or "additional definitive proxy soliciting materials and Rule 14(a)(12) materials." See for example: https://www.sec.gov/edgar/browse/?CIK=320193&owner=exclude

The defa14a.htm (proxy statements) are submitted a few times a year by SEC regulation and most CIK subdirectories do not contain one.

But some CIK subdirectories each year do contain a file such as: https://www.sec.gov/Archives/edgar/data/320193/000130817925000009/aapl4359751-defa14a.htm and in the aapl4359751-defa14a.htm file we find the Schedule 14A proxy statement that includes names of the members of the board of directors.

This last item can be parsed to extract board member names; all publicly held companies are required to submit this information, but how standard the format is unclear.

The example above appears to be too recent and the full report to shareholders is not included, but for an older proxy the full report can be found: https://www.sec.gov/Archives/edgar/data/320193/000130817925000008/aapl4359751-def14a.htm. The report includes board of director biographies, and other officials, and plenty of other

information. It can be parsed to obtain board member bios, and perhaps chief operating officer names.

Of course, this is Apple and not all shareholder reports are likely to be as nicely formatted.

In summary,

- given a company name:
- we obtain the CKC vi the SEC's form interface that we hijack;
- we check all the subdirectories of the CKC to search them for defa14a forms;
- If we find a defa14a form we extract the board of director names;
- If we find a shareholder report we can extract bios and names of executives.

These tasks can be accomplished by fetch/await, together with some html parsing.

The most recent list of board of directors for the given company is our endpoint.

Further processing on these lists is a data science/machine learning/artificial intelligence project.

ChatGPT 4o
Share

JN

You said:
how do I find the board members of apple inc.?

ChatGPT said:
ChatGPT

As of January 2025, Apple's Board of Directors comprises the following members:

- **Arthur D. Levinson**: Chairman of the Board; Founder and CEO of Calico.
- **Tim Cook**: CEO of Apple Inc.
- **Wanda Austin**: Former President and CEO of The Aerospace Corporation.
- **Alex Gorsky**: Former Executive Chair of Johnson & Johnson.
- **Andrea Jung**: President and CEO of Grameen America.
- **Monica Lozano**: Former President and CEO of College Futures Foundation.
- **Ronald D. Sugar**: Former Chair and CEO of Northrop Grumman Corporation.
- **Susan L. Wagner**: Co-founder and Director of BlackRock.

This information is available on Apple's official Leadership page.

Apple

In January 2024, Apple announced that former Vice President Al Gore and former Boeing CFO James Bell would retire from the board, effective February 2024. Subsequently, Wanda Austin was nominated to join the board.

Fox Business

For the most current information, you can visit Apple's official Leadership page.

Apple
Sources
4o

You said:

Is this information available at data.sec.gov?

ChatGPT said:

ChatGPT

Yes, you can find information about the board members of Apple Inc. (or any publicly traded company) on the U.S. Securities and Exchange Commission's (SEC) website at data.sec.gov or the SEC's EDGAR database.

To locate information on Apple's board members via the SEC:

1. **Go to the EDGAR database** (https://www.sec.gov/edgar/searchedgar/companysearch.html).
2. **Search for Apple Inc.**:
   - Use Apple's official company name, *Apple Inc.*, or its SEC Central Index Key (CIK): `0000320193`.
3. **Look for the Proxy Statement (DEF 14A)**:
   - The proxy statement is filed annually and contains detailed information about the company's board of directors, executive compensation, and governance practices.
4. **Open the latest DEF 14A filing**:
   - Review the document for sections about board members. This will provide names, roles, biographies, and committee memberships.

Using the SEC filings ensures you're accessing official and up-to-date information directly from the company's regulatory submissions.

4o

You said:

I see the proxy statements. How do I extract the names of board members and officers from these statements?

ChatGPT said:

ChatGPT

To extract the names of board members and officers from proxy statements (DEF 14A) efficiently, you can follow these steps:

1. Open the Proxy Statement

- Access the DEF 14A document from the SEC's EDGAR database.
- Open the filing and look for sections titled:
    - "Board of Directors"
    - "Corporate Governance"
    - "Executive Officers and Directors"
    - "Election of Directors" (usually near the beginning).

---

2. Search for Relevant Sections

Use the search function (Ctrl+F or Cmd+F) to quickly locate:

- **Board Members**: Search for keywords like "Director," "Board of Directors," or "Nominees for Election."
- **Executive Officers**: Search for "Executive Officers," "Management," or "Compensation Discussion and Analysis."

---

3. Identify Names and Roles

- Look for tables or lists that summarize the information.
- Extract:
    - **Names** of directors and officers.
    - **Roles** (e.g., Chair, CEO, CFO).
    - Committee memberships (e.g., Audit, Compensation, Nominating Committees).

---

4. Copy and Organize Information

- Copy the relevant names and roles into a structured format such as a spreadsheet for easy reference.

---

5. Automate Extraction (Optional)

If you are handling multiple proxy statements or want to streamline the process:

- **Use Text Parsing Tools**:
    1. Tools like Python (with libraries such as `pdfplumber`, `PyPDF2`, or `BeautifulSoup` for HTML filings) can automate the extraction.
- **Steps for Automation**:
    1. Download the proxy statement as a PDF or HTML file.
    2. Use a script to extract sections containing keywords like "Board of Directors" or "Executive Officers."
    3. Parse the text to isolate names, roles, and related information.

Example Python Code

Here's a basic example of extracting names and roles from a proxy statement:

python
Copy code

```
import re

# Load the proxy statement text
with open('proxy_statement.txt', 'r') as file:
    text = file.read()

# Search for board member and officer information
board_section = re.findall(r"Board of Directors.*?\\n\\n(.*?)\\n\\n",
text, re.DOTALL)
executive_section = re.findall(r"Executive
Officers.*?\\n\\n(.*?)\\n\\n", text, re.DOTALL)

# Extract names (example pattern, adjust based on the document
structure)
names = re.findall(r"[A-Z][a-z]+ [A-Z][a-z]+", text)

# Display results
print("Board Members and Officers:")
for name in names:
    print(name)
```

This approach allows you to handle documents programmatically and efficiently. If you need help implementing automation, feel free to ask!

4o

You said:
give me javascript code to do this

ChatGPT said:
ChatGPT

Here's an example of JavaScript code that extracts names of board members and officers from a proxy statement in text format. It uses regular expressions and is designed for text files or HTML content:

javascript
Copy code

```
// Function to extract board members and officers from proxy statement
text
function extractBoardMembersAndOfficers(proxyText) {
    // Regular expressions to find board member and officer sections
    const boardSectionRegex = /Board of Directors([\s\S]*?)\n\n/g;
    const executiveSectionRegex = /Executive Officers([\s\S]*?)\n\n/g;

    // Match and extract relevant sections
    const boardSectionMatch = proxyText.match(boardSectionRegex);
    const executiveSectionMatch =
proxyText.match(executiveSectionRegex);

    // Combine sections for processing
    const combinedSections = (boardSectionMatch ||
[]).concat(executiveSectionMatch || []).join('\n');

    // Regular expression to extract names (simple format for
demonstration)
    const nameRegex =
/\b[A-Z][a-z]+\s[A-Z][a-z]+(?:\s[A-Z][a-z]+)?\b/g;

    // Extract names from the combined text
    const names = [...new Set(combinedSections.match(nameRegex))]; //
Use Set to remove duplicates
```

```
    return names;
}

// Example usage with a sample proxy statement text
const proxyStatementText = `
Board of Directors
John Doe - Chairman
Jane Smith - CEO
Mark Johnson - Director

Executive Officers
Anna White - CFO
Michael Brown - COO
`;

// Extract names
const names = extractBoardMembersAndOfficers(proxyStatementText);

// Display results
console.log('Board Members and Officers:', names);
```

Steps to Use

1. **Prepare the Proxy Statement**:
    - Ensure the proxy statement is in plain text or HTML format.
    - If the file is in PDF, convert it to text using a tool like `pdf2txt`.
2. **Run the Code**:
    - Copy the proxy statement content into the `proxyStatementText` variable or load it dynamically from a file.
    - Run the script in a Node.js environment or in the browser (if loading the text dynamically).
3. **Extend the Regex (if needed)**:
    - Modify the `nameRegex` to better suit the document's naming conventions or format.

Output

The script will extract names such as:

less

Copy code

```
Board Members and Officers: ['John Doe', 'Jane Smith', 'Mark Johnson',
'Anna White', 'Michael Brown']
```

Let me know if you need help customizing this further!

4o
Is this conversation helpful so far?
O
ChatGPT can make mistakes. Check important info.
?
ChatGPT 4o
Share

JN

You said:

js list files in directory "dir"

ChatGPT said:

ChatGPT

To list files in a directory in JavaScript, you can use Node.js's built-in `fs` module. Here's an example:

javascript
Copy code

```
const fs = require('fs');
const path = require('path');

// Define the directory
const directoryPath = path.join(__dirname, 'dir');

// Read files from the directory
fs.readdir(directoryPath, (err, files) => {
  if (err) {
    console.error('Error reading directory:', err);
    return;
  }

  console.log('Files in directory:');
  files.forEach(file => {
```

```
    console.log(file);
  });
});
```

Explanation:

1. **`fs.readdir`**: Asynchronously reads the contents of the directory.
2. **`path.join`**: Ensures the directory path is constructed correctly, regardless of the operating system.
3. **Error handling**: The callback checks if there was an error, like if the directory doesn't exist.
4. **`files`**: The `files` array contains the names of all files and folders in the directory.

If you want only files (excluding subdirectories), you can add a filter using `fs.stat` or `fs.promises.stat` to check if an item is a file:

javascript
Copy code

```
const fs = require('fs');
const path = require('path');

const directoryPath = path.join(__dirname, 'dir');

fs.readdir(directoryPath, (err, files) => {
  if (err) {
    console.error('Error reading directory:', err);
    return;
  }

  console.log('Files only:');
  files.forEach(file => {
    const filePath = path.join(directoryPath, file);
    fs.stat(filePath, (err, stats) => {
      if (err) {
        console.error('Error getting stats:', err);
        return;
      }

      if (stats.isFile()) {
```

```
        console.log(file);
      }
    });
  });
});
```

For Promise-based Approach

Use `fs.promises` for cleaner, async/await-style code:

javascript
Copy code

```
const fs = require('fs').promises;
const path = require('path');

const listFiles = async (directoryPath) => {
  try {
    const files = await fs.readdir(directoryPath);
    console.log('Files in directory:');
    for (const file of files) {
      const filePath = path.join(directoryPath, file);
      const stats = await fs.stat(filePath);
      if (stats.isFile()) {
        console.log(file);
      }
    }
  } catch (err) {
    console.error('Error:', err);
  }
};

listFiles(path.join(__dirname, 'dir'));
```

These methods let you list and filter files in the directory efficiently.

4o

You said:

javascript list the files in this URL: const filingUrl = `${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/-/g, '')}`;

ChatGPT said:

ChatGPT

To list files in a URL (such as a web directory), JavaScript alone cannot do this directly. Most web servers do not allow directory listing for security reasons. However, if the server supports directory listing and provides an HTML page or an API with the file listing, you can retrieve and parse it using JavaScript.

Steps:

1. Fetch the contents of the URL.
2. Parse the response to extract file names if directory listing is enabled.
3. Handle errors if the server does not expose the directory listing.

Here's an example using `fetch` to get the contents of a URL and extract file names:

Example Code:

javascript

Copy code

```
const SEC_BASE_URL = "https://www.sec.gov";
const cik = "0000320193"; // Example CIK
const accessionNumber = "0000320193-23-000052"; // Example accession number
const filingUrl = `${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/-/g, '')}`;

const listFilesInUrl = async (url) => {
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`);
    }
    const text = await response.text();

    // Parse file names from HTML response
    const fileNames = [];
```

```
    const regex = /<a href="([^"]+)"/g; // Matches href links in the
directory
    let match;
    while ((match = regex.exec(text)) !== null) {
      fileNames.push(match[1]);
    }

    console.log('Files in directory:');
    fileNames.forEach(file => console.log(file));
  } catch (error) {
    console.error('Error fetching or parsing directory:', error);
  }
};

listFilesInUrl(filingUrl);
```

Explanation:

1. `fetch`: Used to retrieve the content of the URL.
2. **Regex (`<a href="([^"]+)"`)**: Extracts links from the HTML response, assuming the server provides an HTML directory listing.
3. **Directory Listing**: Many servers disable directory listing, so this approach will only work if the server explicitly allows it.

Limitations:

- If the server does not support directory listing, this will not work.
- In such cases, you need to check if the server offers an API (like a JSON response) for accessing file details.

If you're dealing with the SEC's EDGAR system, it typically doesn't provide directory listings in this way. You might need to analyze the structure of their pages or use their official API for structured access to the data.

4o
O
ChatGPT can make mistakes. Check important info.
?

Chat history

ChatGPT 4o
Share

JN

You said:
I want to find a free API to access records about a company's board member from sec.gov
ChatGPT said:
ChatGPT
The U.S. Securities and Exchange Commission (SEC) provides various free APIs and resources that can be used to access information about companies and their board members.

Below is a guide to help you access this information:

1. SEC EDGAR API

- Description: The SEC's EDGAR system provides public access to financial statements and other filings made by companies. You can use it to retrieve information about a company's board members, usually found in proxy statements (Form DEF 14A) or annual reports (Form 10-K).
- Access:
  - Visit the SEC EDGAR API documentation.
- Endpoints to Use:
  - Company Filings: `https://data.sec.gov/submissions/CIK[CIK_NUMBER].json`
    - Replace `[CIK_NUMBER]` with the Central Index Key (CIK) of the company. You can find a company's CIK using the CIK Lookup tool.
  - Search Filings: Proxy statements (DEF 14A) contain information about board members and executive officers. Use the search API to find relevant documents.

2. Information Found in Proxy Statements (DEF 14A)

- Proxy statements often include:
  - Names and biographical details of board members.
  - Compensation of executives and directors.
  - Share ownership of officers and directors.

3. Steps to Access Board Member Records

- Step 1: Search for the company's filings.
  - Use the search endpoint to locate Form DEF 14A or Form 10-K for the relevant period.
- Step 2: Parse the JSON or HTML response for the filings.
  - Look for the "board of directors" or "executive officers" section.
- Step 3: Extract and analyze the required information.

4. User-Agent Requirement

The SEC requires a user-agent string in API requests. Ensure you include this header in your HTTP requests:

http

Copy code

```
User-Agent: [Your Name] [Your Email Address] [Your Organization]
```

5. Rate Limits

The SEC imposes rate limits on API usage to prevent overloading its systems. Make sure to review their API documentation for the latest details.

Example Python Code

Here’s an example of how to fetch a company's filings using Python:

python

Copy code

```
import requests headers = { 'User-Agent': 'Your Name your.email@example.com Your Organization' } # Replace with the company's CIK number cik_number = '0000320193' # Example: Apple's CIK url = f'https://data.sec.gov/submissions/CIK{cik_number}.json' response = requests.get(url, headers=headers) if response.status_code == 200: data = response.json() # Access filings filings = data.get('filings', {}).get('recent', {}) print(filings) else: print(f"Failed to fetch data: {response.status_code}")
```

This approach will help you gather information on board members via SEC filings programmatically. If you'd like further help refining this process, let me know!

You said:
I want to find a free javascript API to access records about a company's board member from sec.gov
ChatGPT said:
ChatGPT
If you're looking for a JavaScript-compatible API to access records about a company's board members from the SEC's EDGAR system, you can use the SEC EDGAR API, which is publicly available and free. Below is an outline of how you can do this:

---

1. API Endpoint

The SEC EDGAR system provides access to company filings via JSON endpoints. Relevant information about board members is typically found in proxy statements (DEF 14A) or annual reports (10-K).

Endpoints to Use:

- Company Submissions:
  `https://data.sec.gov/submissions/CIK[CIK_NUMBER].json`

Where to Get the CIK Number:

- Use the SEC's CIK Lookup Tool to find the CIK for a given company.

---

2. How to Use with JavaScript

Here's an example of how you can fetch and parse data from the SEC EDGAR API using JavaScript in a Node.js or browser environment.

Example Code

javascript

Copy code

```
const fetchCompanyData = async (cik) => { const url =
`https://data.sec.gov/submissions/CIK${cik}.json`;
 const headers = { 'User-Agent':
 'Your Name your.email@example.com Your Organization', };
 try { const response = await fetch(url, { headers });
 if (response.ok) { const data = await response.json(); console.log('Company
Data:', data);
 // Access recent filings const filings = data.filings.recent; console.log('Recent
Filings:', filings);
 // Look for proxy statements (DEF 14A) or other relevant documents } else {
console.error(`Failed to fetch data. Status: ${response.status}`); } } catch
(error) { console.error('Error fetching data:', error); } };
 // Example usage with Apple's CIK number const appleCIK = '0000320193';
fetchCompanyData(appleCIK);
```

---

3. Parsing DEF 14A Filings

To extract specific board member details:

1. Identify DEF 14A filings in the `filings.recent` array.
2. Parse the `accessionNumber` field to construct the filing URL.
    - Filing URL format: `https://www.sec.gov/Archives/edgar/data/[CIK]/[ACCESSION_NUMBER]-index.html`
3. Scrape or analyze the HTML content for board member details.

---

4. Key Notes

- Rate Limits: The SEC has strict rate limits for API usage. Avoid making frequent requests in a short time frame.

- User-Agent Requirement: Always include a custom `User-Agent` header in your requests, as required by the SEC.
- CORS Issues: If you're using this in a browser and encounter CORS errors, consider using a proxy server or fetching the data server-side.

---

5. Libraries to Simplify Work

To streamline the process, you can use libraries like:

- Node.js `fetch` or `axios`: For making HTTP requests.
- Cheerio: For parsing HTML content from DEF 14A filings.

Would you like further guidance on parsing the proxy statements or building a specific feature using this API?

You said:
yes. write javascript to query the API for board member names, board member bios, and board member transactions.
ChatGPT said:
ChatGPT
To query the SEC EDGAR API for board member names, bios, and transactions, you can use JavaScript. The steps include:

1. Fetch Company Submissions: Identify the DEF 14A filing for board member names and bios.
2. Extract Board Member Details: Parse the DEF 14A filing to get names and bios (may require HTML parsing).
3. Fetch Transactions: Look for Form 4 filings (filed by insiders) to track board member transactions.

Here's a complete JavaScript implementation:

Full Example Code

javascript

Copy code

```
const fetch = require('node-fetch'); // Use this in Node.js. In browsers, native `fetch` works. const SEC_BASE_URL = 'https://data.sec.gov'; const USER_AGENT = 'Your Name your.email@example.com Your Organization'; // Update this! // Function to fetch company data const fetchCompanyData = async (cik) => { const url = `${SEC_BASE_URL}/submissions/CIK${cik}.json`; const headers = { 'User-Agent': USER_AGENT }; try { const response = await fetch(url, { headers }); if (response.ok) { const data = await response.json(); return data; } else { console.error(`Failed to fetch company data. Status: ${response.status}`); return null; } } catch (error) { console.error('Error fetching company data:', error); return null; } }; // Function to fetch and parse board member names and bios const fetchBoardMembers = async (cik) => { const companyData = await fetchCompanyData(cik); if (!companyData) return; const filings = companyData.filings.recent; const filingIndex = filings.form.findIndex((form) => form === 'DEF 14A'); // Proxy Statement if (filingIndex === -1) { console.error('No DEF 14A filings found.'); return; } const accessionNumber = filings.accessionNumber[filingIndex]; const filingUrl = `${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/-/g, '')}/index.html`; console.log('DEF 14A Filing URL:', filingUrl); // For DEF 14A parsing, you would need to scrape the HTML for board member information console.log('Note: You need to scrape the DEF 14A filing to get names and bios.'); }; // Function to fetch insider transactions (Form 4 filings) const fetchInsiderTransactions = async (cik) => { const companyData = await fetchCompanyData(cik); if (!companyData) return; const filings = companyData.filings.recent; const insiderFilings = filings.form .map((form, index) => (form === '4' ? filings.accessionNumber[index] : null)) .filter(Boolean); if (insiderFilings.length === 0) { console.error('No Form 4 filings found.'); return; } console.log('Recent Insider Transactions:'); insiderFilings.forEach((accessionNumber) => { const transactionUrl = `${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/-/g,
```

```
'')}/index.html`; console.log(transactionUrl); }); }; // Main function to orchestrate the queries const main = async () => { const cik = '0000320193'; // Replace with the CIK of your target company (e.g., Apple) console.log('Fetching board members...'); await fetchBoardMembers(cik); console.log('\nFetching insider transactions...'); await fetchInsiderTransactions(cik); }; main();
```

Explanation of Code

1. Fetch Company Data:
   - Queries the SEC's `/submissions/CIK[CIK_NUMBER].json` endpoint for recent filings.
2. Board Member Names & Bios:
   - Looks for DEF 14A filings, where information about board members is usually included.
   - Outputs the URL to the DEF 14A filing. You'll need to scrape this document for names and bios using tools like Cheerio or Puppeteer.
3. Board Member Transactions:
   - Looks for Form 4 filings (filed by insiders for transactions).
   - Outputs URLs to recent Form 4 filings.

Additional Notes

- Scraping DEF 14A: DEF 14A filings are in HTML format. You can use tools like `cheerio` or `puppeteer` to extract board member names and bios.
- Parsing Form 4 Filings: These filings are in structured XML and can be parsed directly to extract transaction details.

Would you like help writing the scraping code for DEF 14A or parsing Form 4 filings? Let me know!

You said:
yes, I would like javascript code for scraping the DEF 14A filings to extract board member names and bios
ChatGPT said:
ChatGPT

Here’s an example of how to scrape the DEF 14A filings to extract board member names and bios using JavaScript. This example uses Puppeteer, a headless browser automation tool, to load and parse the HTML of the DEF 14A filings.

Requirements

1. Install Puppeteer:
2. bash
3. Copy code
4. `npm install puppeteer`
5. Update your script to include the CIK of the company whose DEF 14A you want to parse.

Full Code for Scraping DEF 14A

javascript

Copy code

```
const puppeteer = require('puppeteer'); const SEC_BASE_URL =
'https://www.sec.gov'; // Function to scrape board members from a DEF 14A filing
const scrapeBoardMembers = async (filingUrl) => { const browser = await
puppeteer.launch(); const page = await browser.newPage(); try { // Navigate to the
filing URL await page.goto(filingUrl, { waitUntil: 'load' });
console.log(`Accessing DEF 14A filing at: ${filingUrl}`); // Extract board member
information const boardMembers = await page.evaluate(() => { const boardData = [];
const elements = Array.from(document.querySelectorAll('p, td, span'));
elements.forEach((element) => { const text = element.innerText.trim(); // Look for
patterns indicating board members and their bios if
(text.match(/Director|Board|Chair|CEO|Biography/i)) { boardData.push(text); } });
return boardData; }); console.log('Board Member Details Extracted:');
```

```
boardMembers.forEach((info, idx) => console.log(`${idx + 1}. ${info}`)); await browser.close(); return boardMembers; } catch (error) { console.error('Error scraping DEF 14A:', error); await browser.close(); } }; // Function to find and scrape DEF 14A filing for a company const scrapeDef14AFiling = async (cik) => { const companyDataUrl = `${SEC_BASE_URL}/submissions/CIK${cik}.json`; const headers = { 'User-Agent': 'Your Name your.email@example.com Your Organization' }; try { // Fetch company submissions const response = await fetch(companyDataUrl, { headers }); const data = await response.json(); const filings = data.filings.recent; // Find the most recent DEF 14A filing const filingIndex = filings.form.findIndex((form) => form === 'DEF 14A'); if (filingIndex === -1) { console.error('No DEF 14A filings found.'); return; } const accessionNumber = filings.accessionNumber[filingIndex]; const filingUrl = `${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/-/g, '')}/index.html`; console.log(`DEF 14A Filing URL: ${filingUrl}`); // Scrape board members from the filing const boardMembers = await scrapeBoardMembers(filingUrl); console.log(`Extracted ${boardMembers.length} board member entries.`); } catch (error) { console.error('Error fetching company data or filings:', error); } }; // Main Function const main = async () => { const cik = '0000320193'; // Example: Apple's CIK console.log('Scraping DEF 14A filing for board member details...'); await scrapeDef14AFiling(cik); }; main();
```

Explanation

1. Puppeteer:
   - Puppeteer is used to open and navigate to the DEF 14A filing page.
   - It selects relevant `<p>`, `<td>`, and `<span>` elements to extract possible board member names and bios.
2. Text Matching:
   - The code identifies board member information by matching text patterns like `Director`, `Board`, `Chair`, `CEO`, or `Biography`.
   - This is a heuristic; you can fine-tune it for better results.
3. File Navigation:

- The script identifies the most recent DEF 14A filing using the EDGAR JSON data.
- It constructs the filing URL to load the HTML page.

4. Results:
    - Extracted board member names and bios are printed to the console.

Output Example

When run, the script will produce output like:

markdown

Copy code

```
Accessing DEF 14A filing at:
https://www.sec.gov/Archives/edgar/data/0000320193/0001193125-23-055432-index.html
Board Member Details Extracted: 1. Tim Cook, Chief Executive Officer 2. Arthur
Levinson, Chairman of the Board 3. Al Gore, Director, former Vice President of the
United States ... Extracted 10 board member entries.
```

Notes

- Improving Accuracy: Parsing DEF 14A filings requires understanding the structure of the document. The script uses simple heuristics, but more advanced text parsing may be needed for edge cases.
- Rate Limits: Use the `User-Agent` header to comply with SEC guidelines and avoid being blocked.
- Enhancements: Consider using natural language processing (NLP) libraries to better identify bios and filter irrelevant text.

Would you like help refining this code further or adding advanced parsing capabilities?

You said:
yes, I could I add advanced parsing capabilities?
ChatGPT said:
ChatGPT

To add advanced parsing capabilities for extracting board member names and bios from DEF 14A filings, you can incorporate Natural Language Processing (NLP) and pattern-matching techniques to make the extraction more accurate.

Here's how you can enhance the existing script:

Enhancements for Advanced Parsing

1. Use an NLP Library:
   - Integrate an NLP library like Compromise or Natural for JavaScript.
   - Use these libraries to identify names and structured bios.
2. Text Cleaning and Tokenization:
   - Clean up raw text by removing extraneous whitespace, special characters, and HTML tags.
   - Tokenize the content into sentences for better analysis.
3. Pattern Matching for Names:
   - Names often follow patterns like `First Last`, `First M. Last`, or similar.
   - Use regular expressions to identify these patterns.
4. Extract Role Information:
   - Look for roles like `Director`, `CEO`, `Chairman`, etc., near the names.
   - Use proximity analysis to link names with their roles.

Updated Script with Advanced Parsing

Here's the updated script with these capabilities:

Step 1: Install Necessary Libraries

Install `compromise` for NLP and `cheerio` for better HTML parsing:

bash

Copy code

```
npm install compromise cheerio
```

Step 2: Updated Code

javascript

Copy code

```
const puppeteer = require('puppeteer'); const nlp = require('compromise'); const cheerio = require('cheerio'); const SEC_BASE_URL = 'https://www.sec.gov'; // Advanced function to process and extract names and bios const extractBoardMembers = (htmlContent) => { const $ = cheerio.load(htmlContent); const rawText = $('body').text(); const cleanText = rawText.replace(/\s+/g, ' ').trim(); // Split text into sentences const sentences = cleanText.split(/[.?!]\s/); const boardMembers = []; sentences.forEach((sentence) => { const doc = nlp(sentence); // Extract potential person names const people = doc.people().out('array'); if (people.length > 0) { // Check for roles like "Director", "Chair", etc. const roles = /(Director|Chair|CEO|President|Officer|Board)/i; if (roles.test(sentence)) { boardMembers.push({ name: people.join(', '), bio: sentence.trim(), }); } } }); return boardMembers; }; // Function to scrape and parse DEF 14A filing const scrapeBoardMembers = async (filingUrl) => { const browser = await puppeteer.launch(); const page = await browser.newPage(); try { // Navigate to the filing URL await page.goto(filingUrl, { waitUntil: 'load' }); console.log(`Accessing DEF 14A filing at: ${filingUrl}`); // Extract HTML content of the filing const htmlContent = await page.content(); // Extract board member details using NLP const boardMembers = extractBoardMembers(htmlContent); console.log('Board Member Details Extracted:'); boardMembers.forEach((member, idx) => { console.log(`${idx + 1}. Name: ${member.name}`); console.log(` Bio: ${member.bio}`); }); await browser.close(); return boardMembers; } catch (error) { console.error('Error scraping DEF 14A:', error); await browser.close(); } }; // Function to find and scrape DEF 14A filing for a company const scrapeDef14AFiling = async (cik) => { const companyDataUrl = `${SEC_BASE_URL}/submissions/CIK${cik}.json`; const headers = { 'User-Agent': 'Your Name your.email@example.com Your Organization' }; try { // Fetch company submissions const response = await fetch(companyDataUrl, { headers }); const data = await response.json(); const filings = data.filings.recent; // Find the most
```

```
recent DEF 14A filing const filingIndex = filings.form.findIndex((form) => form === 'DEF 14A'); if (filingIndex === -1) { console.error('No DEF 14A filings found.'); return; } const accessionNumber = filings.accessionNumber[filingIndex]; const filingUrl = `${SEC_BASE_URL}/Archives/edgar/data/${cik}/${accessionNumber.replace(/-/g, '')}/index.html`; console.log(`DEF 14A Filing URL: ${filingUrl}`); // Scrape board members from the filing const boardMembers = await scrapeBoardMembers(filingUrl); console.log(`Extracted ${boardMembers.length} board member entries.`); } catch (error) { console.error('Error fetching company data or filings:', error); } }; // Main Function const main = async () => { const cik = '0000320193'; // Example: Apple's CIK console.log('Scraping DEF 14A filing for board member details...'); await scrapeDef14AFiling(cik); }; main();
```

How It Works

1. HTML Parsing with Cheerio:
   - The filing's HTML content is parsed to extract raw text using `cheerio`.
2. NLP with Compromise:
   - `compromise` identifies names in the text.
   - Sentences are scanned for roles (e.g., `Director`, `CEO`) and linked with names.
3. Role-Bio Association:
   - Sentences containing roles are tagged as potential bios for the identified names.

Example Output

When run, the script produces output like:

yaml

Copy code

```
Accessing DEF 14A filing at:
https://www.sec.gov/Archives/edgar/data/0000320193/0001193125-23-055432-index.html
```